

KN-STORE

Plataforma E-commerce · Spring Boot + React + MongoDB

PLAN DE TRABAJO

Backend · Nucleo Transaccional e Infraestructura

Desarrollador	Santiago
Rol	Backend · Nucleo transaccional e infraestructura
Rama de trabajo	santiago
Responsable tecnico	Joseph (integracion main)
Fecha de emision	25 de julio de 2026
Version	v1.0

1. Contexto y alcance

KN-Store ya tiene el núcleo del e-commerce construido: catálogo, carrito persistente, checkout transaccional y panel de cliente. Tu responsabilidad es el núcleo transaccional del backend y la infraestructura que lo sostiene: seguridad consolidada, datos de prueba automáticos, transacciones reales en MongoDB, direcciones completas, totales consistentes, búsqueda del catálogo, operación administrativa de pedidos/envíos y los tests que prueban que no hay sobreventa.

Tus tareas son el cimiento del equipo: Laura (pagos y facturación) y Nicolás (frontend) dependen de tus entregas para iniciar sus fases 2 y 3. El orden de este plan está pensado para que nunca te detengas esperando a nadie: las 6 primeras tareas no tienen dependencias.

2. Resumen de tareas

ID	Tarea	Prioridad	RF / RNF	Depende de	Inicio
S-01	Protección de rutas backend	ALTA	RF-005	—	Inmediato
S-02	Seed automático del catálogo	ALTA	RF-011..030	—	Inmediato
S-03	MongoDB replica set	ALTA	RNF-023	—	Inmediato
S-04	Direcciones + predeterminada	ALTA	RF-037..041	—	Inmediato
S-05	Recálculo subtotal carrito	MEDIA	RF-043/048	—	Inmediato
S-06	Busqueda full-text	MEDIA	RF-024/025	—	Inmediato
S-07	Totales consistentes checkout	MEDIA	RF-047/048	S-05	Fase 2
S-08	Endpoints admin pedidos/envíos	MEDIA	RF-052/061..065	S-03, S-07	Fase 2
S-09	Tests checkout y concurrencia	ALTA	RNF-023/024	S-03/05/07	Fase 3

3. Orden de implementación y detalle

FASE 1 · INICIO INMEDIATO — sin dependencias (S-01 » S-06)

S-01 · Estabilizar la protección de rutas del backend

Prioridad: **ALTA** RF/RNF: RF-005 Depende de: Ninguna

OBJETIVO

Consolidar y versionar la configuración de seguridad. Hoy existen 5 archivos con cambios sin commitar (SecurityConfiguration y application*.yaml) que dejan la protección de rutas en estado inestable.

IMPLEMENTACIÓN

- Revisar el diff pendiente de SecurityConfiguration.java (regla /api/users restringida a ADMIN/MANAGER) y de los application*.yaml (prod, secret-samples, test).
- Confirmar que el catálogo público siga abierto: GET /api/categorias/**, /api/subcategorias/**, /api/productos/**, /api/marcas/**.
- Commitar con mensaje convencional: fix(security): consolidar reglas de acceso de /api/users y configuración.
- Ejecutar la suite de seguridad: SecurityUtilsUnitTest, ResourceAccessServiceTest, TokenAuthenticationIT, AuthenticationIntegrationTest.
- Documentar la matriz final ruta » rol en arquitectura.md (sección de seguridad).

ARCHIVOS CLAVE

src/main/java/com/mycompany/knstore/config/SecurityConfiguration.java

src/main/resources/config/application.yaml · arquitectura.md*

CRITERIOS DE ACEPTACION

- Working tree limpio: ningún cambio de seguridad pendiente de commit.
- Suite de seguridad completamente en verde.
- Matriz de rutas y roles publicada en arquitectura.md.

Puede iniciar: De inmediato. Es la primera tarea de tu rama.

Desbloquea: [Base de confianza para todos los endpoints nuevos del equipo.](#)

S-02 · Carga de datos automática del catálogo (seed)

Prioridad: **ALTA** **RF/RNF:** RF-011 a RF-030 (datos) · RNF-016 **Depende de:** Ninguna

OBJETIVO

Que el sistema levante con datos demo sin pasos manuales. Hoy el catálogo se siembra con un script Node externo (scripts/seed-demo-data.js) que hay que ejecutar a mano contra la API.

IMPLEMENTACION

- Crear un ChangeUnit de Mongock (config/dbmigrations/CatalogSeedMigration.java) que corra después de InitialSetupMigration.
- Sembrar: Categoría IVA 19%; marcas (Nike, Adidas, Puma); 4 categorías con subcategorías (Deportivo, Casual, Formal); ~30 productos con slug, ProductoPrecio (compra/venta) y ProductoInventario (stock, bodega).
- Portar la lógica idempotente (createOrUpdate por slug/nombre) desde scripts/seed-demo-data.js al ChangeUnit.
- Activar el seed solo en desarrollo (propiedad knstore.seed.catalog=true en application-dev.yml) para no contaminar producción.
- Verificar con GET /api/productos?size=5 inmediatamente después de arrancar.

ARCHIVOS CLAVE

src/main/java/com/mycompany/knstore/config/dbmigrations/ · scripts/seed-demo-data.js (referencia) · application-dev.yml

CRITERIOS DE ACEPTACION

- El backend levanta en dev con el catálogo visible en la tienda sin ejecutar scripts externos.
- Seed idempotente: una re-ejecución no duplica documentos.
- Producción arranca sin datos demo.

Puede iniciar: De inmediato.

Desbloquea: [Datos de prueba para todo el equipo \(backend y frontend\).](#)

S-03 · MongoDB replica set para transacciones reales

Prioridad: **ALTA** **RF/RNF:** RNF-023 **Depende de:** Ninguna

OBJETIVO

Hacer que el @Transactional del checkout funcione de verdad. El docker actual levanta MongoDB standalone y las transacciones multi-documento se ignoran silenciosamente.

IMPLEMENTACION

- Migrar src/main/docker/mongodb.yml a un replica set de un nodo (mongod --replSet rs0 + contenedor de inicialización con rs.initiate()), o promover mongodb-cluster.yml como archivo por defecto.
- Actualizar services.yml y la URI en application-dev.yml agregando ?replicaSet=rs0&retryWrites=true.
- Documentar el levantamiento en README.md (sección de desarrollo).
- Prueba de humo: un checkout que falla a mitad (stock insuficiente en el segundo ítem) no debe dejar rastro en la base de datos.

ARCHIVOS CLAVE

src/main/docker/mongodb.yml · src/main/docker/services.yml · src/main/resources/config/application-dev.yml ·

*README.md***CRITERIOS DE ACEPTACION**

- `rs.status()` muestra un nodo PRIMARY.
- Una excepcion a mitad del checkout revierte pedido, items, pago y stock (rollback real).

Puede iniciar: De inmediato. Es infraestructura aislada, no toca codigo de negocio.

Desbloquea: [L-03 \(pasarela de Laura requiere transacciones\)](#) y [S-09 \(tests transaccionales\)](#).

S-04 · Direcciones: campos de envio y predeterminada atomica

Prioridad: **ALTA** **RF/RNF:** RF-037 a RF-041 **Depende de:** Ninguna

OBJETIVO

Completar el modelo de direcciones para envios reales y eliminar el N+1 que el frontend ejecuta hoy para marcar una direccion predeterminada.

IMPLEMENTACION

- Agregar a la entidad Direccion: telefonoContacto (obligatorio), destinatario (obligatorio), codigoPostal (opcional) y predeterminada (boolean, default false).
- Actualizar DireccionDTO, mapper, knstore.jdl y .jhipster/Direccion.json para mantener el generador sincronizado.
- ChangeUnit Mongock: inicializar predeterminada=false en las direcciones existentes.
- Crear PATCH `/api/direcciones/{id}/predeterminada`: en una sola operacion desmarca las demas direcciones de la cuenta (updateMulti) y marca la indicada.
- Mantener la verificacion de propiedad via ResourceAccessService (el CLIENTE solo toca sus direcciones).
- Publicar el contrato del endpoint (request/response de ejemplo) en el PR para Nicolas.

ARCHIVOS CLAVE

domain/Direccion.java · service/dto/DireccionDTO.java · web/rest/DireccionResource.java · service/impl/DireccionServiceImpl.java · knstore.jdl

CRITERIOS DE ACEPTACION

- Una sola llamada PATCH deja exactamente una direccion predeterminada por cuenta.
- DireccionResourceIT actualizado y en verde (incluye el caso de otro propietario » 403).

Puede iniciar: De inmediato.

Desbloquea: [N-04 \(Nicolas actualiza el formulario y consume el PATCH atomico\)](#).

S-05 · Carrito: recalcu automatico de subtotal

Prioridad: **MEDIA** **RF/RNF:** RF-043 · RF-048 **Depende de:** Ninguna

OBJETIVO

Que subtotal y fechaActualizacion del carrito se mantengan solos. Hoy CarritoServiceImpl es un CRUD generico y el total se desincroniza al modificar items.

IMPLEMENTACION

- En ItemCarritoServiceImpl (save/update/delete) recalcular: subtotal del item = cantidad x precioUnitario; subtotal del carrito = suma de items; actualizar fechaActualizacion.
- Si el cliente no envia precioUnitario, tomarlo del ProductoPrecio vigente del producto.
- Devolver el carrito con sus totales en las respuestas de creacion/edicion de items.

ARCHIVOS CLAVE

service/impl/ItemCarritoServiceImpl.java · service/impl/CarritoServiceImpl.java · web/rest/ItemCarritoResource.java

CRITERIOS DE ACEPTACION

- Crear, editar o eliminar un item actualiza el subtotal del carrito sin llamadas extra del cliente.
- Tests de servicio cubren los tres casos.

Puede iniciar: De inmediato (puede ir en paralelo con S-04).

Desbloquea: S-07 (los totales consistentes parten del carrito bien calculado).

S-06 · Búsqueda full-text de productos (endpoint paginado)

Prioridad: **MEDIA** **RF/RNF:** RF-024 · RF-025 **Depende de:** Ninguna

OBJETIVO

Reemplazar el filtro en memoria del frontend. Los índices \$text ya están declarados en Producto; falta exponer el endpoint.

IMPLEMENTACION

- Crear GET /api/productos/search?q=&page=&size= usando TextCriteria sobre el índice \$text de Producto.
- Abrir el endpoint en SecurityConfiguration (permitAll, igual que el resto del catálogo público).
- Excluir productos inactivos y ordenar por score de texto.
- Responder Page<ProductoDTO> estandar con header X-Total-Count.
- Publicar el contrato en el PR para Nicolas.

ARCHIVOS CLAVE

repository/ProductoRepository.java · service/impl/ProductoServiceImpl.java · web/rest/ProductoResource.java · config/SecurityConfiguration.java

CRITERIOS DE ACEPTACION

- Búsqueda 'tenis deportivo' devuelve resultados paginados en menos de 500 ms (P95) con el catálogo seed.
- ProductoResourceIT cubre el endpoint (caso con y sin resultados).

Puede iniciar: De inmediato (paralelo con S-04/S-05).

Desbloquea: N-06 (SearchPage consume el endpoint).

FASE 2 · TRAS FASE 1 INTEGRADA EN MAIN (S-07 » S-08)

S-07 · Checkout: reglas de envío e IVA consistentes con el frontend

Prioridad: **MEDIA** **RF/RNF:** RF-047 · RF-048 **Depende de:** S-05

OBJETIVO

Que el total que ve el cliente y el que cobra el backend sean exactamente el mismo número. Hoy el frontend da envío gratis desde \$150.000 y el backend siempre cobra \$9.900.

IMPLEMENTACION

- calcularCostoEnvio en CheckoutService: envío gratis si el subtotal es mayor o igual a \$150.000; si no, tarifa por TipoServicioEnvio (tabla vigente: ESTANDAR \$9.900, etc.).
- Mantener el IVA por categoriaIva de cada producto y documentar la fórmula (base gravable, porcentaje, redondeo HALF_UP a 2 dígitos).
- Exponer POST /api/pedidos/preview que devuelva el desglose (subtotal, IVA, envío, total) sin crear el pedido, para que el checkout del frontend lo consuma.
- Publicar la tabla de reglas y el contrato del preview en el PR.

ARCHIVOS CLAVE

service/CheckoutService.java · web/rest/PedidoResource.java

CRITERIOS DE ACEPTACION

- El mismo carrito produce el mismo total en frontend y backend.
- Envío gratis aplicado correctamente a ambos lados del umbral de \$150.000.

Puede iniciar: Cuando S-05 este mergeado a main.

Desbloquea: N-05 (Nicolas consume los totales) y L-03 (Laura integra la pasarela sobre un checkout estable). Al cerrarla, la propiedad de CheckoutService pasa a Laura.

S-08 · Endpoints admin: estados de pedido y gestion de envios

Prioridad: **MEDIA** **RF/RNF:** RF-052 · RF-061 · RF-062 · RF-063 · RF-065 **Depende de:** S-03 · S-07

OBJETIVO

Dar a ADMIN/MANAGER endpoints de operacion real, no solo el CRUD generico de JHipster.

IMPLEMENTACION

- PATCH /api/pedidos/{id}/estado con maquina de estados: PENDING » CONFIRMED » SHIPPED » DELIVERED; CANCELLED solo desde PENDING o CONFIRMED.
- PATCH /api/envios/{id}/tracking para asignar transportadora y numeroRastreo (RF-061).
- PATCH /api/envios/{id}/estado (PENDING » IN_TRANSIT » DELIVERED) y PATCH /api/envios/{id}/devolucion (RF-062, RF-063).
- GET /api/envios/pendientes?page=&size= para la bandeja logistica (RF-065).
- Proteger con @PreAuthorize(ADMIN, MANAGER) y registrar cada transicion en el historial de estados de Laura (L-01): acordar la firma del servicio con ella antes de implementar.

ARCHIVOS CLAVE

web/rest/PedidoResource.java · web/rest/EnvioResource.java · service/impl/PedidoServiceImpl.java · service/impl/EnvioServiceImpl.java

CRITERIOS DE ACEPTACION

- Las transiciones invalidas devuelven 400 con mensaje claro (RNF-013).
- Cada transicion queda registrada en el historial con fecha y actor.
- Tests de integracion de ambos recursos en verde.

Puede iniciar: Cuando S-03 y S-07 esten mergeados, y tras acordar con Laura el servicio de historial (L-01).

Desbloquea: N-09 (panel admin de Nicolas).

FASE 3 · CIERRE DE CALIDAD (S-09)**S-09 · Tests de checkout y concurrencia de stock**

Prioridad: **ALTA** **RF/RNF:** RNF-023 · RNF-024 **Depende de:** S-03 · S-05 · S-07

OBJETIVO

Cubrir la logica mas critica del negocio, que hoy no tiene un solo test.

IMPLEMENTACION

- CheckoutServiceIT: flujo feliz (pedido + items + pago + envio + factura + carrito vaciado), stock insuficiente, precio modificado desde el carrito, carrito vacio y usuario sin cuenta.
- Test de concurrencia: ExecutorService con 20 hilos comprando el mismo producto con stock 10 » el stock nunca queda negativo y se venden exactamente 10 unidades.
- Verificar el rollback transaccional sobre el replica set de S-03.
- Documentar el resultado del test de concurrencia en el PR.

ARCHIVOS CLAVE

src/test/java/com/mycompany/knstore/service/CheckoutServiceIT.java · src/test/java/com/mycompany/knstore/web/rest/PedidoResourceIT.java

CRITERIOS DE ACEPTACION

- Suite verde con Testcontainers.
- El test de concurrencia demuestra que no hay sobreventa.

Puede iniciar: Cuando S-03, S-05 y S-07 esten mergeados.

Desbloquea: Confianza para que Laura refactorice el checkout en L-03.

4. Mapa de dependencias del equipo

Leyenda: A » B significa que B necesita a A mergeado en main para iniciar.

- S-03 (replica set) + S-07 (checkout consistente) » L-03 (pasarela de pagos)
- L-03 » L-04 (reembolsos) y L-05 (facturacion real)
- S-04 (direcciones) » N-04 (formulario y predeterminada)
- S-06 (busqueda full-text) » N-06 (SearchPage)
- S-07 (reglas de totales) » N-05 (checkout consume backend)
- L-03 (estados de pago) » N-07 (flujo de pago en UI)
- L-05 (PDF de factura) » N-08 (boton de descarga)
- S-08 (endpoints admin) + L-04 (reembolsos) » N-09 (panel admin)
- L-01 (historial de estados) es insumo de S-08: Santiago registra transiciones con el servicio de Laura.

5. Reglas de coordinacion

- Ramas: trabajas exclusivamente en tu rama (santiago). Sincroniza con origin/main al inicio de cada semana (merge, no rebase).
- Orden de integracion a main: santiago » lauraG » Nicolas. El backend publica primero; el frontend solo consume contratos ya integrados en main.
- Contrato primero: quien crea o modifica un endpoint publica en el PR un ejemplo de request/response y avisa al equipo antes de mergear.
- Archivos compartidos (avisa antes de tocarlos): SecurityConfiguration.java, application*.yaml, knstore.jdl, pom.xml, routes.tsx, app.tsx, package.json.
- Propiedad transitoria de CheckoutService.java: Santiago hasta cerrar S-07; despues Laura para L-03. Nadie mas lo modifica mientras tanto.
- Commits en formato convencional (feat/fix/chore/test/docs), como el historial del repositorio.
- Todo PR exige la suite en verde: ./mvnw verify (backend) o npm test (frontend).
- Si dos tareas chocan por dependencia, mergea primero quien bloquea al otro.